To better understand how Strict Loading works, take a look at an example using the Rideshare application, continuing to work with the Vehicles and Vehicle Reservations.

Run the following code in bin/rails console.

ruby/vehicle_reservations_strict_loading.rb

```
vehicles = Vehicle.strict_loading.all

vehicles.each do |vehicle|
  vehicle.vehicle_reservations.first.starts_at
end

# `Vehicle` is marked for strict_loading. The VehicleReservation association
# named `:vehicle_reservations` cannot be lazily loaded.
# (ActiveRecord::StrictLoadingViolationError)
```

This time when you run the code you'll get this error:

Vehicle is marked for strict_loading. The VehicleReservation association named :vehicle_reservations cannot be lazily loaded. (ActiveRecord::StrictLoadingViolationError)

When Strict Loading is enabled, Eager Loading must be performed in order to run this query. Try modifying the query above using the Eager Loading techniques you learned earlier to avoid the Strict Loading exception being raised.

You've now worked through one way of removing the possibility of N+1 queries from your Rails application. What other kinds of query-level optimizations can you make?

Optimizing Individual Active Record Queries

Database connections are a finite resource, and all queries require a database connection to run. Reducing unnecessary queries avoids excessive use of limited database connections which can help the scalability of your database server.

What kinds of changes can you make on an individual query basis?

Limiting the fields needed can help speed up queries. One way to do that in Active Record is to use .select(), which does not generate a SQL query and uses loaded data. (See: *Rails Pluck vs Select and Map/Collect*[9])

Another method is .pluck(), which generates SQL that specifies limited fields.

9. https://www.rubyinrails.com/2014/06/05/rails-pluck-vs-select-map-collect/